

Experiment 05 – Advanced SQL Queries

Sahil Goyal | UID: 24BDA70148

Aim

To write and execute advanced SQL queries involving aggregate functions, conditional (CASE) expressions, joins, grouping, and subqueries to solve practical, real-world database problems.

Objective

- Compute rates/ratios using SQL aggregate functions.
- Handle records that have no matching rows on the joined side.
- Calculate a cancellation rate for a set of valid transactions.
- Combine multiple tables using joins with filtering conditions.
- Use conditional aggregation together with ROUND() and COALESCE().

Question 1

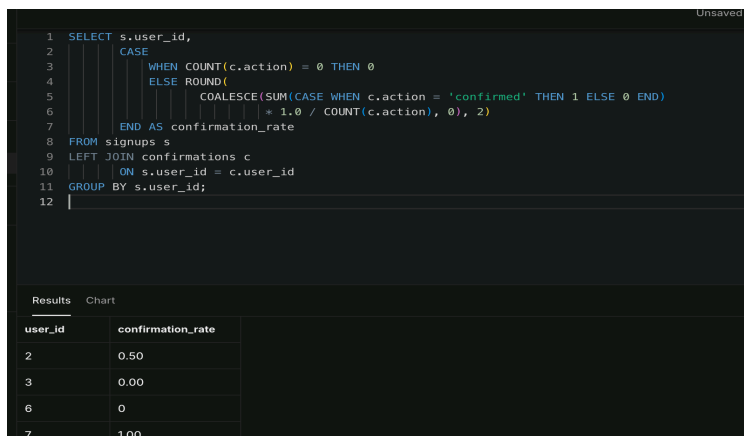
A platform stores every user who has signed up, along with the outcome of any confirmation request sent to that user. Each request is either confirmed or left to expire unconfirmed.

For every registered user, find their confirmation rate — the number of confirmed requests divided by the total requests they received. A user who never received a request should show a rate of 0. Return the user ID with the rate rounded to 2 decimal places, in any order.

Query

```
SELECT s.user_id,
       CASE
         WHEN COUNT(c.action) = 0 THEN 0
         ELSE ROUND(
           COALESCE(SUM(CASE WHEN c.action = 'confirmed' THEN 1 ELSE 0 END)
                     * 1.0 / COUNT(c.action), 0), 2)
       END AS confirmation_rate
FROM signups s
LEFT JOIN confirmations c
      ON s.user_id = c.user_id
GROUP BY s.user_id;
```

Output



```
1 SELECT s.user_id,
2       CASE
3         WHEN COUNT(c.action) = 0 THEN 0
4         ELSE ROUND(
5           COALESCE(SUM(CASE WHEN c.action = 'confirmed' THEN 1 ELSE 0 END)
6                     * 1.0 / COUNT(c.action), 0), 2)
7       END AS confirmation_rate
8 FROM signups s
9 LEFT JOIN confirmations c
10      ON s.user_id = c.user_id
11 GROUP BY s.user_id;
12 |
```

user_id	confirmation_rate
2	0.50
3	0.00
6	0
7	1.00

Question 2

A ride-booking service logs every trip request together with the client and driver involved. Some users are banned from the platform; a trip only counts as valid when both the client and the driver on it are active (not banned). Each valid trip is either completed or cancelled by one of the two parties.

For each date between 2013-10-01 and 2013-10-03, find the cancellation rate among valid trips that day — cancelled trips divided by all valid trips. Return the date and the rate rounded to 2 decimal places.

Query

```
SELECT t.request_at AS day,  
       ROUND(SUM((t.status <> 'completed')::int) * 1.0 / COUNT(*), 2)  
       AS cancellation_rate  
FROM trips t  
LEFT JOIN users c ON t.client_id = c.users_id  
LEFT JOIN users d ON t.driver_id = d.users_id  
WHERE c.banned <> 'Yes'  
      AND d.banned <> 'Yes'  
      AND t.request_at BETWEEN '2013-10-01' AND '2013-10-03'  
GROUP BY t.request_at  
ORDER BY t.request_at;
```

Output

```
1 SELECT t.request_at AS day,  
2      ROUND(SUM((t.status <> 'completed')::int) * 1.0 / COUNT(*), 2)  
3      AS cancellation_rate  
4 FROM trips t  
5 LEFT JOIN users c ON t.client_id = c.users_id  
6 LEFT JOIN users d ON t.driver_id = d.users_id  
7 WHERE c.banned <> 'Yes'  
8      AND d.banned <> 'Yes'  
9      AND t.request_at BETWEEN '2013-10-01' AND '2013-10-03'  
10 GROUP BY t.request_at  
11 ORDER BY t.request_at;  
12
```

Results Chart

day	cancellation_rate
2013-10-01	0.33
2013-10-02	0.00
2013-10-03	1.00

Learning Outcome

- Computed rates and percentages using SQL aggregate functions.
- Used CASE expressions for conditional aggregation.
- Applied LEFT JOIN to retain rows without a matching counterpart.
- Used multiple joins to validate records across related tables.
- Filtered records using multiple conditions and a date range.
- Used ROUND() and COALESCE() to produce cleanly formatted results.